POS Callback Integration — Bug Advisory & Fix Guide

From: Same Day Solution — Technical Team
Date: 21 July 2026
Subject: Incorrect handling of POS transaction callbacks leading to excess retailer payments
Priority: Critical

---

1. Issue Summary

Your system is crediting retailers for **all** incoming POS transaction callbacks, including **failed**, **voided**, and **intermediate (authorized-only)** transactions. This has resulted in excess payments of approximately ₹2–3 Lakhs to your retailers.

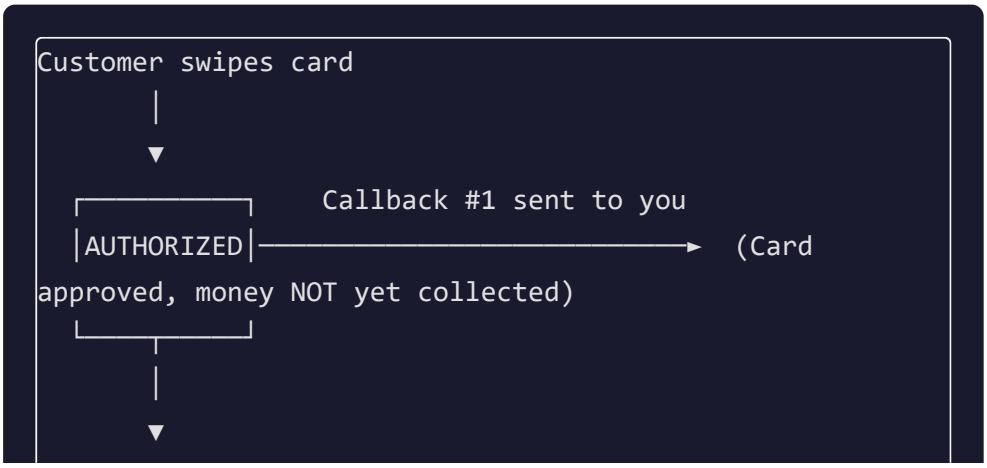
---

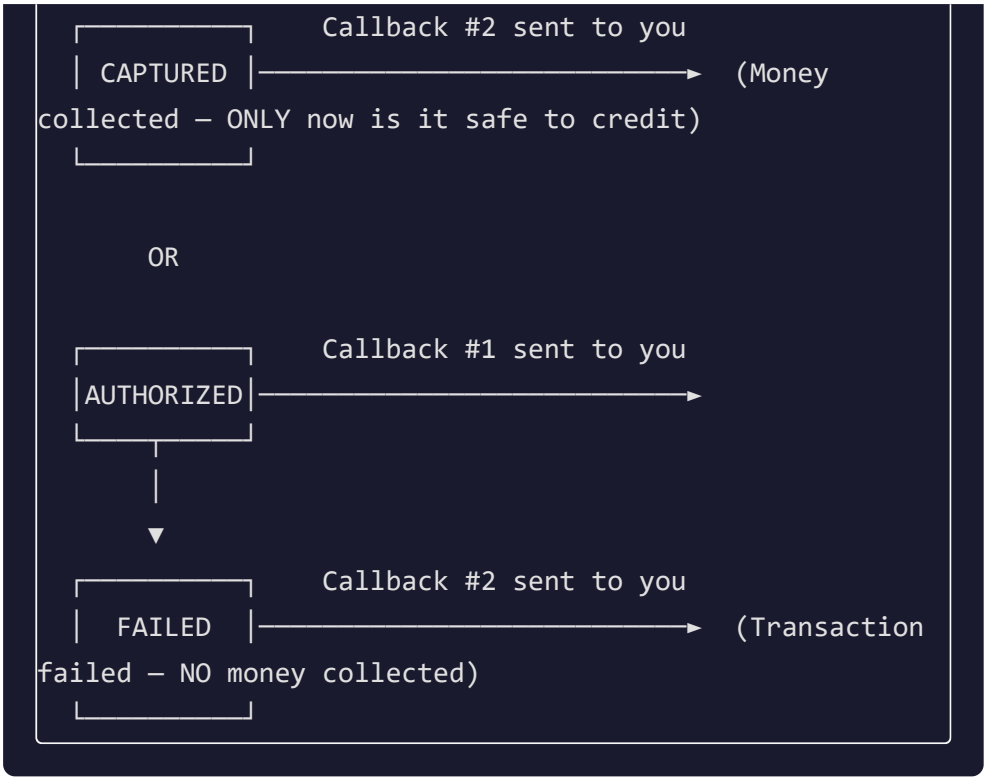
2. How Our Callback System Works

When a POS transaction occurs on any terminal assigned to your account, Razorpay sends us a webhook notification. We process it and **forward the original payload to your registered webhook URL** in real time.

Callback Lifecycle for a Single Transaction

A single card swipe can trigger **multiple callbacks** as the transaction progresses through its lifecycle:





All Possible Status Values

Status	Meaning	Money Received?	Credit Retailer?
`AUTHORIZED`	Card approved, settlement pending	**No**	**No**
`CAPTURED`	Settlement complete, money collected	**Yes**	**Yes**
`FAILED`	Transaction failed	**No**	**No**
`VOIDED`	Transaction cancelled before capture	**No**	**No**
`REFUNDED`	Money returned to customer	**No**	**No** (reverse if already credited)

---

3. Callback Payload Format

Every callback is an HTTP `POST` to your webhook URL with `Content-Type: application/json`. The body is the raw Razorpay POS payload:

```
{
  "txnId": "260216093324974E883523117",
  "tid": "96192813",
  "amount": 10000,
  "status": "CAPTURED",
  "rrNumber": "000000000012",
  "paymentMode": "CARD",
  "paymentCardType": "CREDIT",
  "paymentCardBrand": "VISA",
  "postingDate": "2026-02-16T09:33:26.000+0000",
  "settlementStatus": "PENDING",
  "externalRefNumber": "EZ2026021615031185O8",
  "deviceSerial": "2841157353"
}
```

> ***The `status` field is the most critical field in this payload.***
> *The `amount` field is in **paisa** (divide by 100 for rupees).*

---

4. The Two Bugs in Your Integration

Bug 1: Not Filtering on `status`

Your webhook handler is treating every callback as a successful payment. You **must** check the `status` field and only credit a retailer when `status` is exactly `"CAPTURED"`.

All other statuses (`AUTHORIZED`, `FAILED`, `VOIDED`, `REFUNDED`) must be ignored for crediting purposes.

Bug 2: No Deduplication by `txnId`

A single transaction triggers multiple callbacks (e.g., first `AUTHORIZED`, then `CAPTURED`). If your system credits on both, the retailer gets paid **twice** for the same swipe.

You must use `txnId` as a unique/idempotency key and ensure each transaction is credited only once.

---

5. Recommended Fix

Below is the corrected logic for your webhook handler. Adapt to your language/framework:

Pseudocode

```
RECEIVE callback payload

IF payload.status ≠ "CAPTURED"
    LOG "Ignoring txnId={payload.txnId}, status=
{payload.status}"
    RESPOND 200 OK
    RETURN

IF txnId already exists in your credited_transactions
table
    LOG "Duplicate callback for txnId={payload.txnId},
skipping"
    RESPOND 200 OK
    RETURN

INSERT txnId into credited_transactions table
CREDIT retailer with (payload.amount / 100) rupees

RESPOND 200 OK
```

Node.js Example

```
app.post('/your-webhook-endpoint', async (req, res)
=> {
  const { txnId, status, amount, tid } = req.body;

  // Step 1: Only process CAPTURED transactions
  if (status !== 'CAPTURED') {
    console.log(`Skipped: txnId=${txnId},
status=${status}`);
    return res.status(200).json({ received: true });
  }

  // Step 2: Deduplication check
  const existing = await db.query(
    'SELECT id FROM credited_transactions WHERE
razorpay_txn_id = $1',
    [txnId]
  );

  if (existing.rows.length > 0) {
    console.log(`Duplicate: txnId=${txnId}, already
credited`);
    return res.status(200).json({ received: true,
duplicate: true });
  }

  // Step 3: Record and credit
  await db.query(
    'INSERT INTO credited_transactions
(razorpay_txn_id, amount_paisa, terminal_id,
credited_at) VALUES ($1, $2, $3, NOW())',
    [txnId, amount, tid]
  );

  const amountRupees = amount / 100;
  await creditRetailer(tid, amountRupees);

  console.log(`Credited: txnId=${txnId}, ₹
```

```
${amountRupees}`);
  return res.status(200).json({ received: true,
credited: true });
});
```

Python (Flask) Example

```
@app.route('/your-webhook-endpoint', methods=
['POST'])
def handle_pos_callback():
    data = request.get_json()
    txn_id = data.get('txnId')
    status = data.get('status')
    amount = data.get('amount', 0)

    # Step 1: Only process CAPTURED
    if status != 'CAPTURED':
        return jsonify({'received': True}), 200

    # Step 2: Deduplication
    if
CreditedTransaction.query.filter_by(razorpay_txn_id=txn_
        return jsonify({'received': True,
'duplicate': True}), 200

    # Step 3: Credit retailer
    record =
CreditedTransaction(razorpay_txn_id=txn_id,
amount_paisa=amount)
    db.session.add(record)
    db.session.commit()

    credit_retailer(data.get('tid'), amount / 100)
    return jsonify({'received': True, 'credited':
True}), 200
```

---

6. Recovering the Excess Payments

To identify which transactions were incorrectly credited, use our Transaction API:

```
POST /api/partner/pos-transactions
Headers:
  x-api-key: <your-api-key>
  x-timestamp: <current-unix-ms>
  x-signature: <HMAC signature>

Body:
{
  "status": "FAILED",
  "date_from": "2026-01-01",
  "date_to": "2026-07-21",
  "page": 1,
  "page_size": 100
}
```

This returns all **FAILED** transactions within the date range. Cross-reference these `txnId` values against your credited_transactions table to find the incorrectly paid ones.

Repeat with `"status": "AUTHORIZED"` to find transactions that were authorized but never captured (never settled).

---

7. Important Rules for Callback Handling

Rule	Detail
Always return HTTP 200	Even for ignored/failed processing. Non-200 responses may cause retries.
Only credit on `CAPTURED`	This is the only status that confirms money has been collected.

Deduplicate by `txnId`	Same transaction can trigger 2+ callbacks. Credit only once.
Store `txnId` permanently	Keep a log of every credited `txnId` for reconciliation.
Amount is in paisa	Divide by 100 to get rupees. ₹500 comes as `50000`.
Log everything	Log every callback received with `txnId`, `status`, and `amount` for audit trail.

---

8. Testing Before Going Live

Use our test callback endpoint to verify your handler:

1. We can trigger test callbacks to your webhook URL with different statuses.
2. Confirm your system **ignores** `AUTHORIZED`, `FAILED`, `VOIDED`, `REFUNDED`.
3. Confirm your system **credits only** on `CAPTURED`.
4. Confirm duplicate `CAPTURED` callbacks for the same `txnId` are not double-credited.

Contact our technical team to schedule a test run.

---

9. Checklist Before Resuming Live Operations

- ☐ Webhook handler checks `status === "CAPTURED"` before crediting
- ☐ Deduplication by `txnId` is implemented
- ☐ All historical incorrect credits have been identified and reversed
- ☐ Callback logging is in place for audit trail
- ☐ Test callbacks with all status types have been verified
- ☐ Amount conversion from paisa to rupees is correct

For technical support, contact the Same Day Solution API team.